



CHAPTER # 3

GATE LEVEL MINIMIZATION

Lecturer: Noman Al Hassan
Email: noman44m@hotmail.com

Objective of Lecture

After studying this chapter, you should be able to know

- Introduction.
- Karnaugh Map Method.
 - Two Variable K-Maps.
 - Three Variable K-Maps.
 - Four Variable K-Maps.
- Map Simplification and Manipulation.
- Determination and Selection of Prime Implicants.
- Don't Care Conditions.
- NAND and NOR operations.
- Other two level Implementations.

Introduction

- Boolean expressions can be simplified by algebraic manipulation, such an approach lacks clear regular rules for each succeeding step and it is difficult to determine whether the simplest expression has been achieved.
- In contrast Karnaugh-map (K-map) method provides a straightforward procedure for simplifying Boolean functions.
- K- map is a pictorial method of simplifying **boolean expressions** without using boolean algebra theorems and equation manipulations.
- Logic minimization
 - The Karnaugh map
 - A simple straight forward procedure
 - A pictorial form of a truth table
 - Applicable if the # of variables < 7
- A diagram represent:
 - Each square represents one minterm.
 - **Goal is to draw as big boxes as possible with minimum overlapping.**

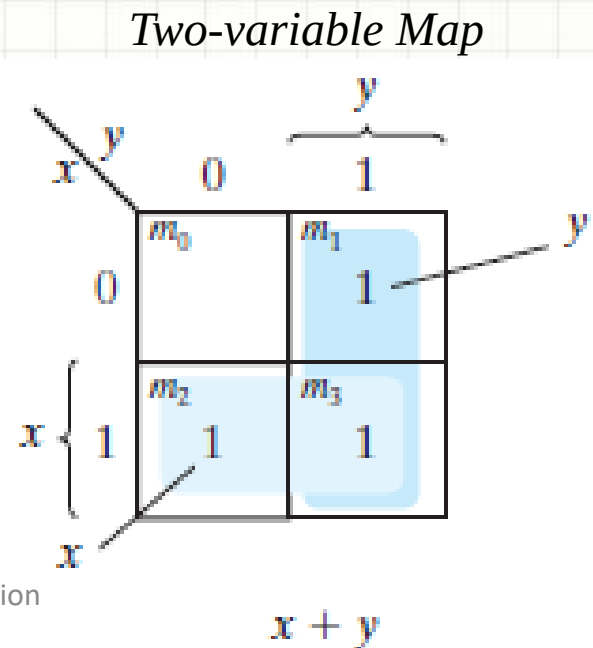
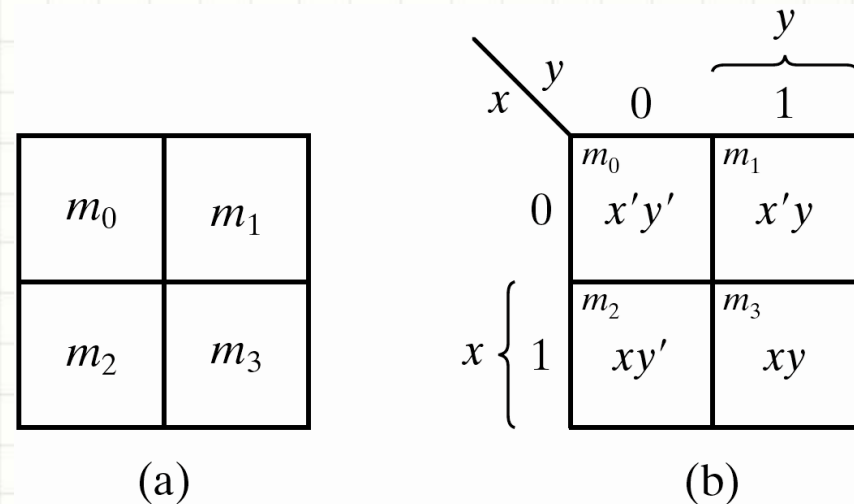
Rules for K-Map

1. No zeros allowed.
2. Groups may be horizontal or vertical, but not diagonal.
3. Groups must contain 1, 2, 4, 8, or in general 2^n cells.
4. Groups should be as large as possible.
5. Every cell must be in at least one group.
6. Overlapping allowed.
7. Wrap around allowed.
8. Fewest number of groups possible.

Two-Variables K-Map

- A two-variables K- map
 - Four minterms
 - $x' = \text{row } 0$
 - $x = \text{row } 1$
 - $y' = \text{column } 0$
 - $y = \text{column } 1$
- A truth table in square diagram
- Example:
- $F = (1, 2, 3)$

$$m_1 + m_2 + m_3 = x'y + xy' + xy$$



A Three-variable Map

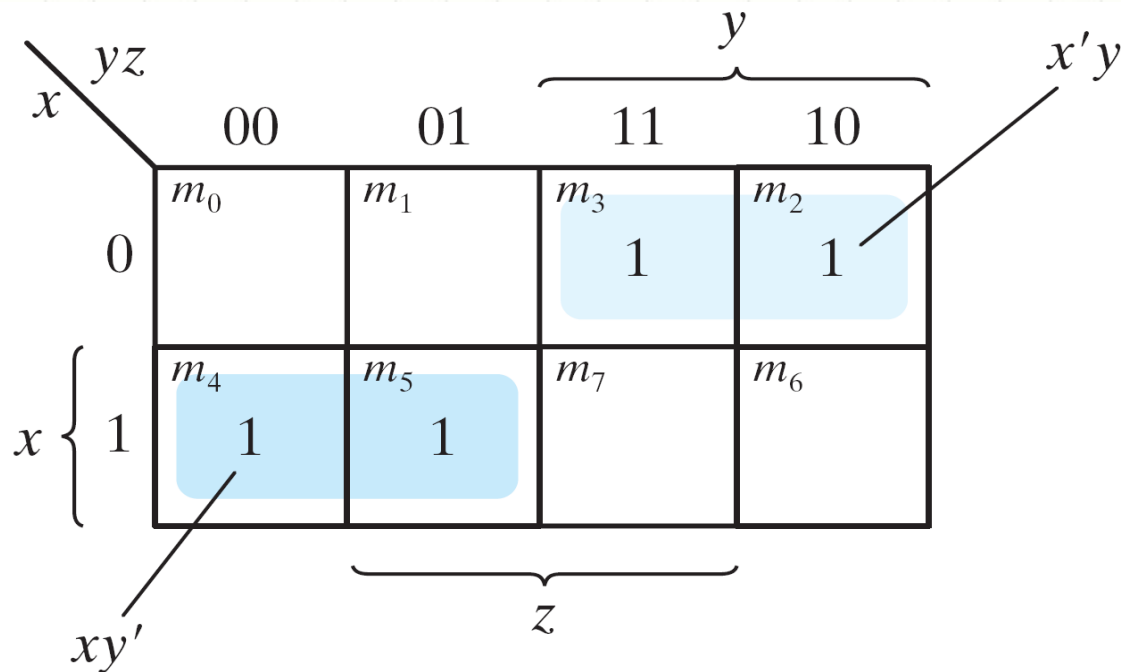
- There are 8 minterms for 3 variables.
- Therefore there are 8 cells in 3 variables K-map.
- The map
 - 8 minterms
 - Combinations of 2, 4 and 8 adjacent squares.
- Any two adjacent squares in the map differ by only one variable
 - m_0 and m_2 (m_4 and m_6) are adjacent

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

		y			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

Example:

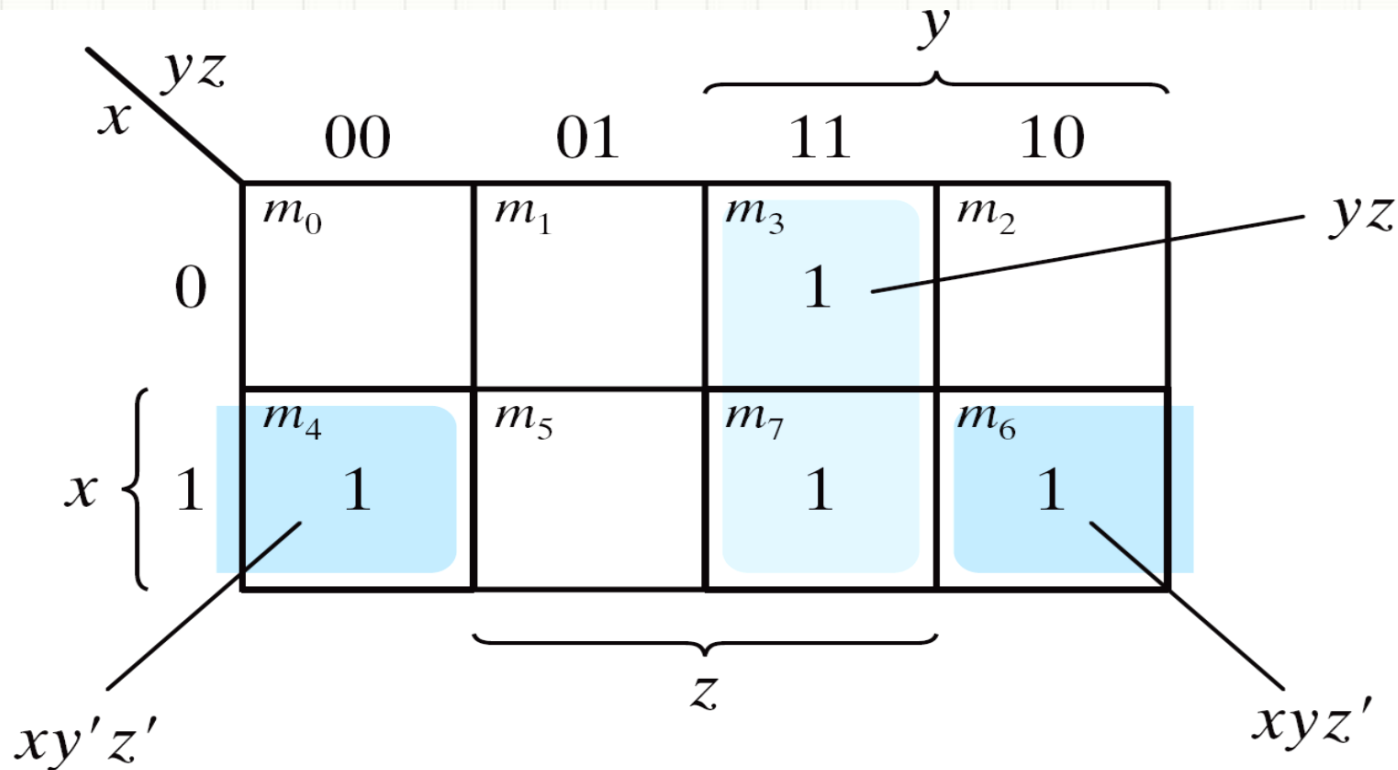
- Simplify the Boolean function
 - $F(x, y, z) = \Sigma(2, 3, 4, 5)$



$$F(x, y, z) = \Sigma(2, 3, 4, 5) = x'y + xy'$$

Example:

- Simplify $F(x, y, z) = \Sigma(3, 4, 6, 7)$
 - $F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$

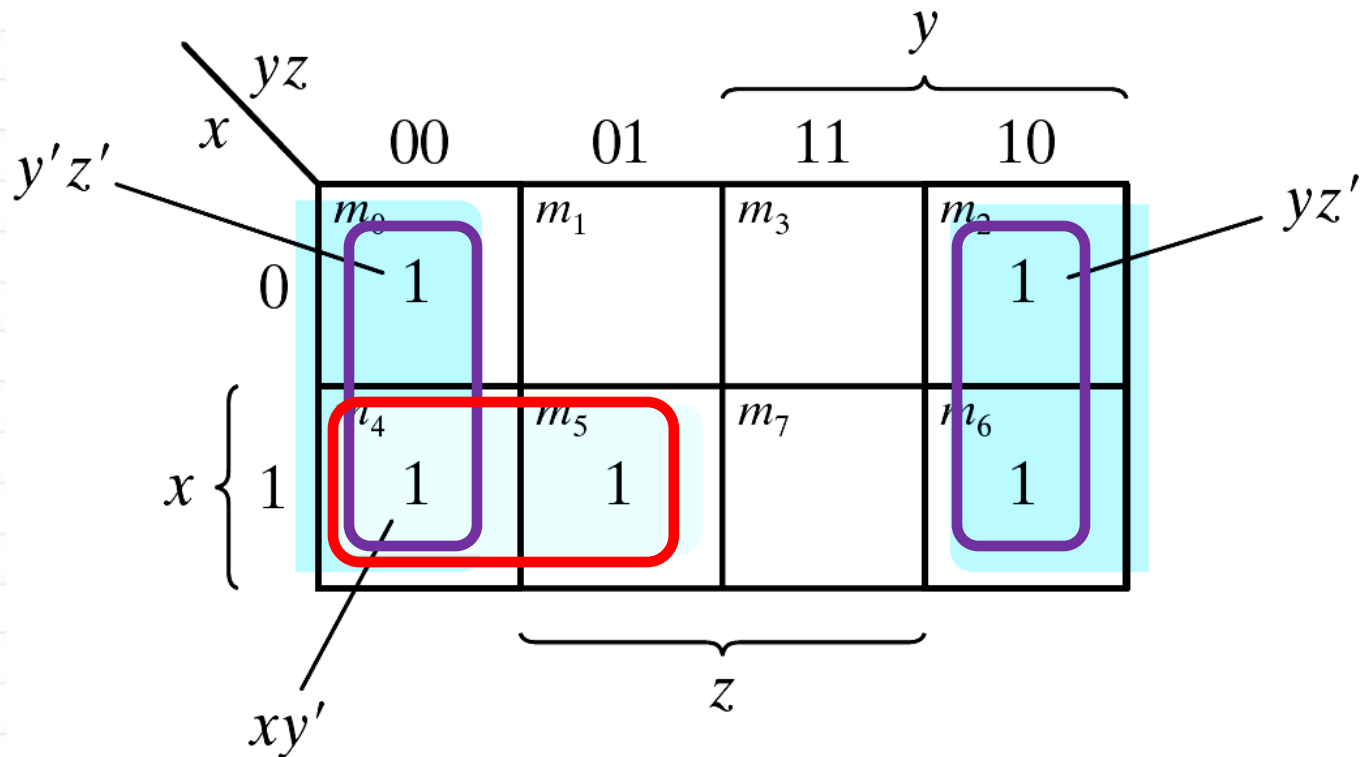


Note: $xy'z' + xyz' = xz'$

$$F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$$

Example:

- Simplify $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$



Note: $y'z' + yz' = z'$

$$F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$$

Four adjacent Squares

– 2, 4, and 8 squares

$$\begin{aligned}
 & m_0 + m_2 + m_4 + m_6 \\
 &= x'y'z' + x'yz' + xy'z' + xyz' \\
 &= x'z'(y' + y) + xz'(y' + y) \\
 &= x'z' + xz' \\
 &= z'
 \end{aligned}$$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

$$\begin{aligned}
 & m_1 + m_3 + m_5 + m_7 \\
 &= x'y'z + x'yz + xy'z + xyz \\
 &= x'z(y' + y) + xz(y' + y) \\
 &= x'z + xz \\
 &= z
 \end{aligned}$$

		y			
		xz	00	01	11
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'
		z			

(a)

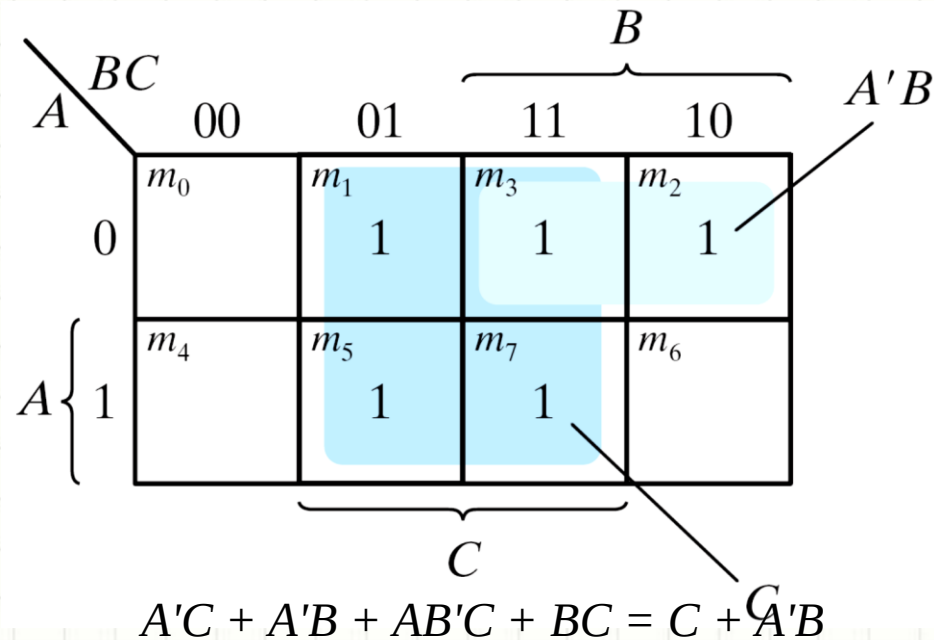
(b)

Figure 3.3 Three-variable Map

Example:

- Example: let $F = A'C + A'B + AB'C + BC$
 - Express it in sum of minterms.
 - Find the minimal sum of products expression.

Ans: $F(A, B, C) = \Sigma(1, 2, 3, 5, 7) = C + A'B$



Comparison

THREE-VARIABLE K-MAP

- One square represents one minterm, giving a term with three literals.
- Two adjacent squares represent a term with two literals.
- Four adjacent squares represent a term with one literal.
- Eight adjacent squares encompass the entire map and produce a function that is always equal to 1.

FOUR-VARIABLE K-MAP

- One square represents one minterm, giving a term with four literals.
- Two adjacent squares represent a term with three literals.
- Four adjacent squares represent a term with two literals.
- Eight adjacent squares represent a term with one literal.
- Sixteen adjacent squares produce a function that is always equal to 1.

Four-Variable Map

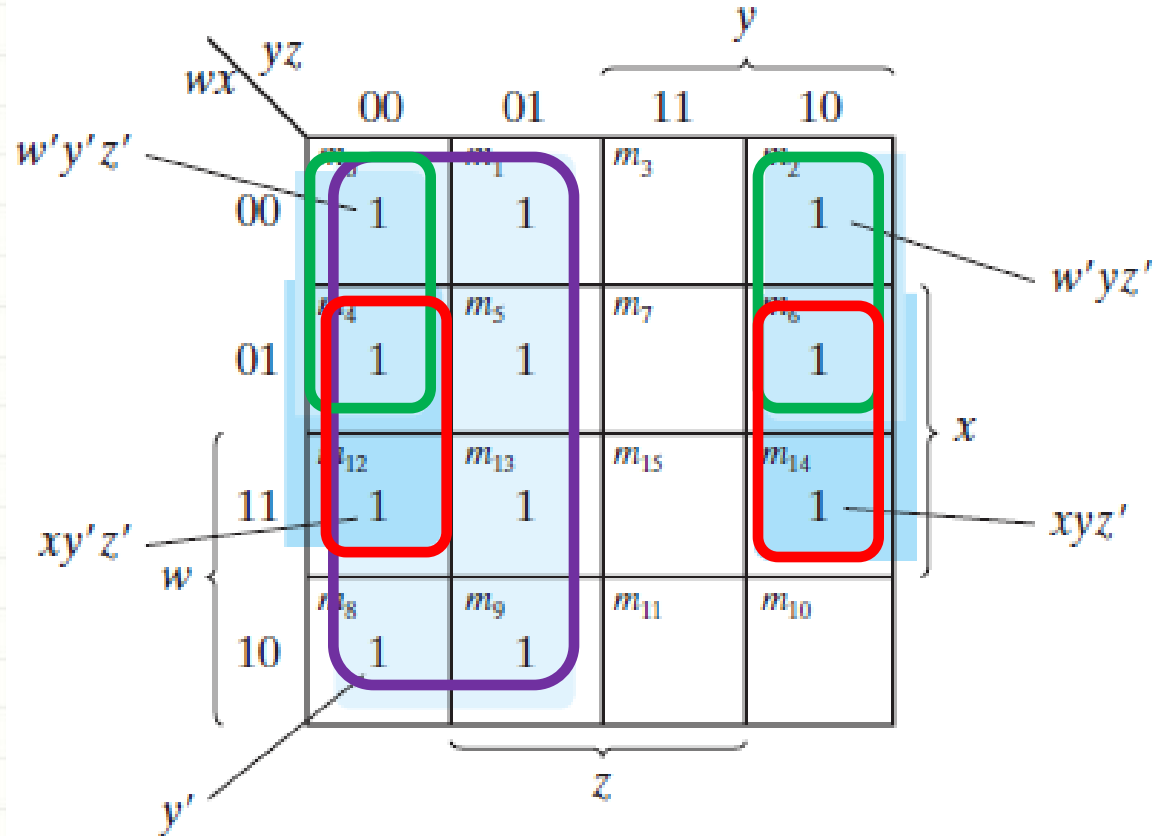
- There are 16 minterms for 4 variables.
- Therefore there are 16 cells in 4 variables K-map.
- The map
 - 16 minterms
 - Combinations of 2, 4, 8, and 16 adjacent squares.
- Any two adjacent squares in the map differ by only one variable
 - m_0 and m_2 (m_4 and m_6) are adjacent

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

		y			
		00	01	11	10
wx	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$

Example:

- Simplify $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$

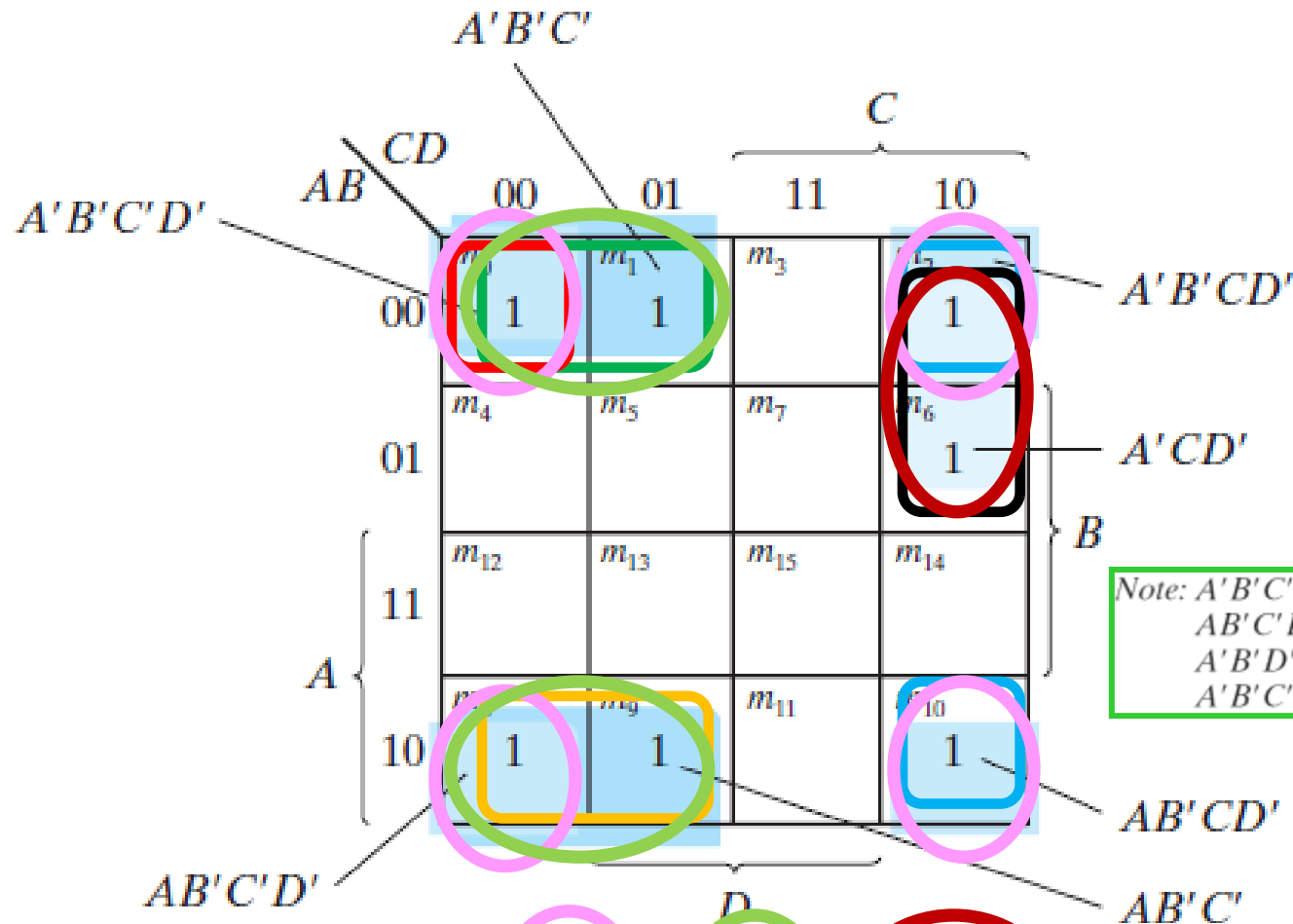


Note: $w'y'z' + w'yz' = w'z'$
 $xy'z' + xyz' = xz'$

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = y' + w'z' + xz'$$

Example:

- Simplify $F = A'B'C' + B'CD' + A'B'C'D' + AB'C' + A'CD'$

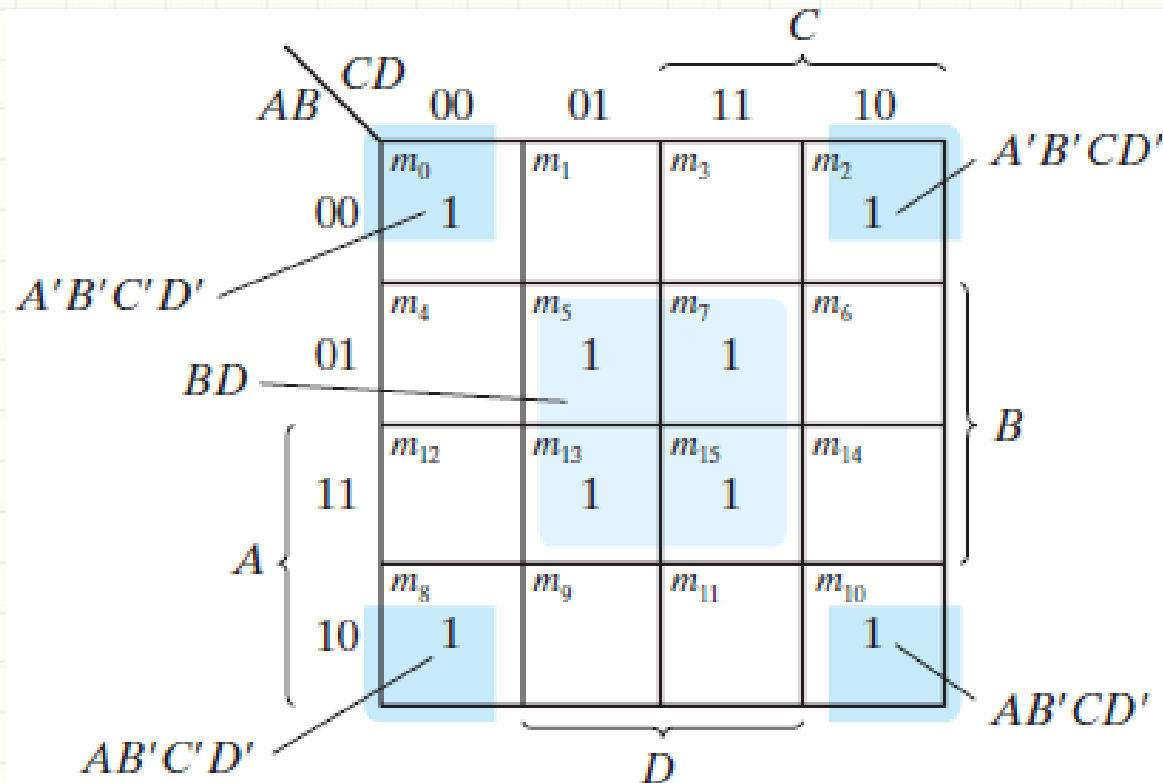


Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

$$B'D' + B'C' + A'CD'$$

Prime Implicants.

- The prime implicants of a function can be obtained from the map by combining all possible maximum numbers of squares.



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$

(a) Essential prime implicants
 BD and $B'D'$

Product of Sums Simplification

- Simplify the following Boolean function into

- (a) sum-of-products form

- (b) product-of-sums form

$$F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$$

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1 1	m_3 0	m_2 1
	01	m_4 0	m_5 1	m_7 0	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 0	m_{14} 0
	10	m_8 1	m_9 1	m_{11} 0	m_{10} 1
		D			

$$(a) \quad F = B'D' + B'C' + A'C'D$$

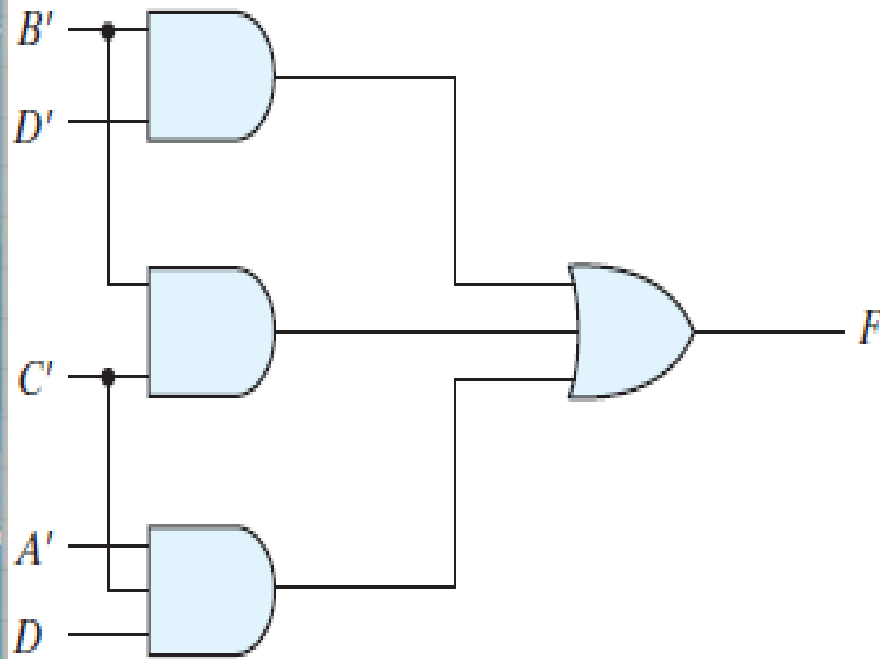
$$(b) \quad F' = AB + CD + BD'$$

Applying DeMorgan's theorem (by taking the dual)

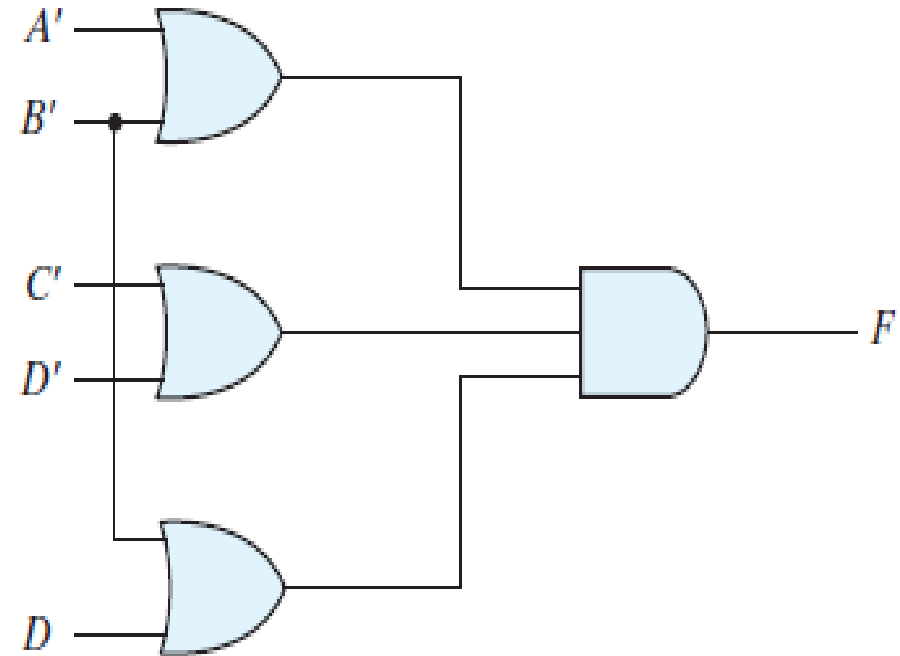
$$(A' + B')(C' + D')(B' + D)$$

$$F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10) = B'D' + B'C' + A'C'D = (A' + B')(C' + D')(B' + D)$$

Two Level Implementation



(a) $F = B'D' + B'C' + A'C'D$



(b) $F = (A' + B')(C' + D')(B' + D)$

Gate implementations of the function

DON'T-CARE CONDITIONS

Simplify the Boolean function

$$F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$$

which has the don't-care conditions

$$d(w, x, y, z) = \Sigma(0, 2, 5)$$

		y			
		00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
w	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

Diagram (a) shows a 4x4 Karnaugh map for the function $F(w, x, y, z)$. The map is labeled with w and x on the left, and y and z on the top and bottom. The cells are labeled m_0 through m_{15} . The function is defined by the 1s in the cells $m_1, m_3, m_7, m_{11}, m_{15}$. The don't-care conditions are marked with 'X' in the cells m_0, m_2, m_5 . The map is grouped into four 2x2 blocks, each labeled with a w value (00, 01, 11, 10) and an x value (0, 1). The groups are labeled $w'x'$, $w'x$, wx , and wx' . The function is simplified to $F = yz + w'x'$.

(a) $F = yz + w'x'$

		y			
		00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
w	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

Diagram (b) shows a 4x4 Karnaugh map for the function $F(w, x, y, z)$. The map is labeled with w and x on the left, and y and z on the top and bottom. The cells are labeled m_0 through m_{15} . The function is defined by the 1s in the cells $m_1, m_3, m_7, m_{11}, m_{15}$. The don't-care conditions are marked with 'X' in the cells m_0, m_2, m_5 . The map is grouped into four 2x2 blocks, each labeled with a w value (00, 01, 11, 10) and an x value (0, 1). The groups are labeled $w'z$, $w'x$, wx , and wx' . The function is simplified to $F = yz + w'z$.

(b) $F = yz + w'z$

$$F(w, x, y, z) = yz + w'x' = \Sigma(0, 1, 2, 3, 7, 11, 15)$$

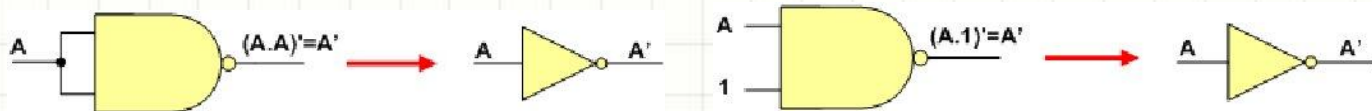
$$F(w, x, y, z) = yz + w'z = \Sigma(1, 3, 5, 7, 11, 15)$$

NAND and NOR Implementation

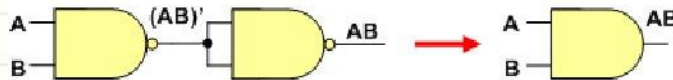
- The NAND and NOR gates are universal gates.
- A universal gate is a gate which can implement any Boolean function without need to use any other gate type.
- Economical and easier to fabricate and are the basic gates used in all IC.
- In fact, an AND gate is typically implemented as a NAND gate followed by an inverter not the other way around.
- Likewise, an OR gate is typically implemented as a NOR gate followed by an inverter not the other way around.

NAND Implementation

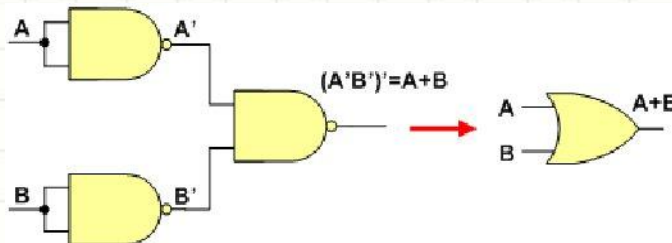
- Implementing an Inverter Using only NAND Gate



- Implementing AND Using only NAND Gates



- Implementing OR Using only NAND Gates



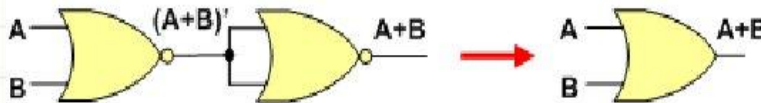
- Thus, the NAND gate is a universal gate since it can implement the AND, OR and NOT functions.

NOR Implementation

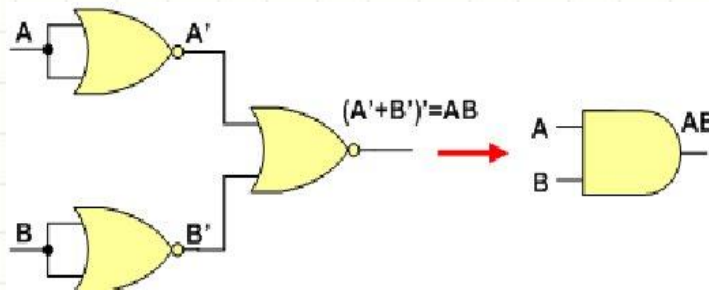
- Implementing an Inverter Using only NOR Gate



- Implementing OR Using only NOR Gates



- Implementing AND Using only NOR Gates



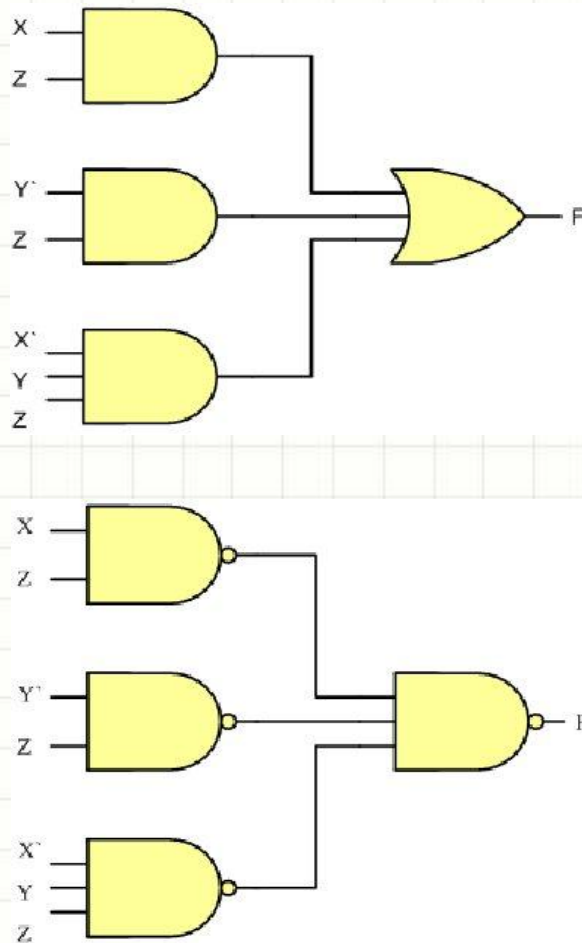
- Thus, the NOR gate is a universal gate since it can implement the AND, OR and NOT functions.

Other Two Level Implementation

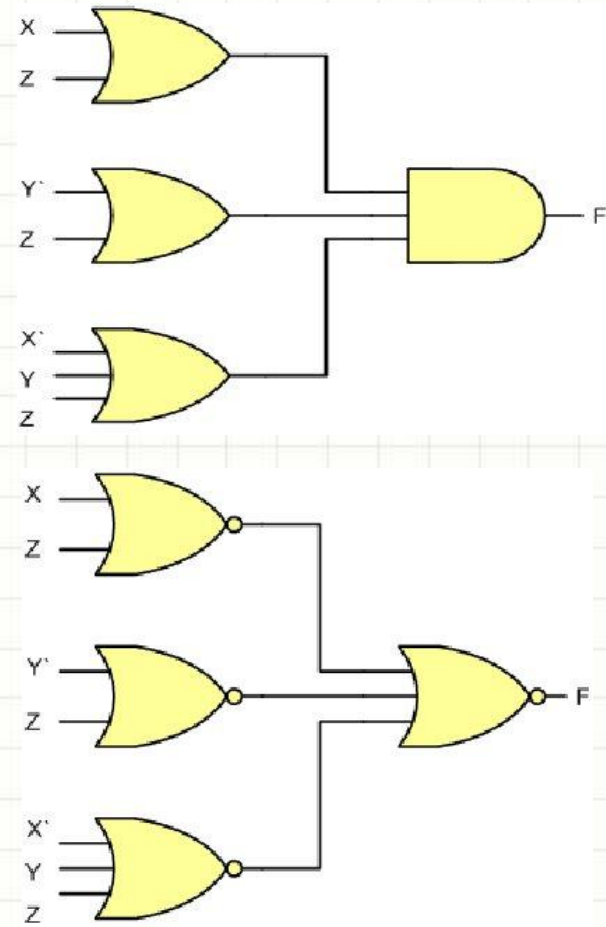
- We have seen in previous chapter that Boolean functions in either SOP or POS forms can be implemented using 2-Level implementations.
- For SOP forms AND gates will be in the first level and OR gate will be in the second level.
- For POS forms OR gates will be in the first level and a single AND gate will be in the second level.

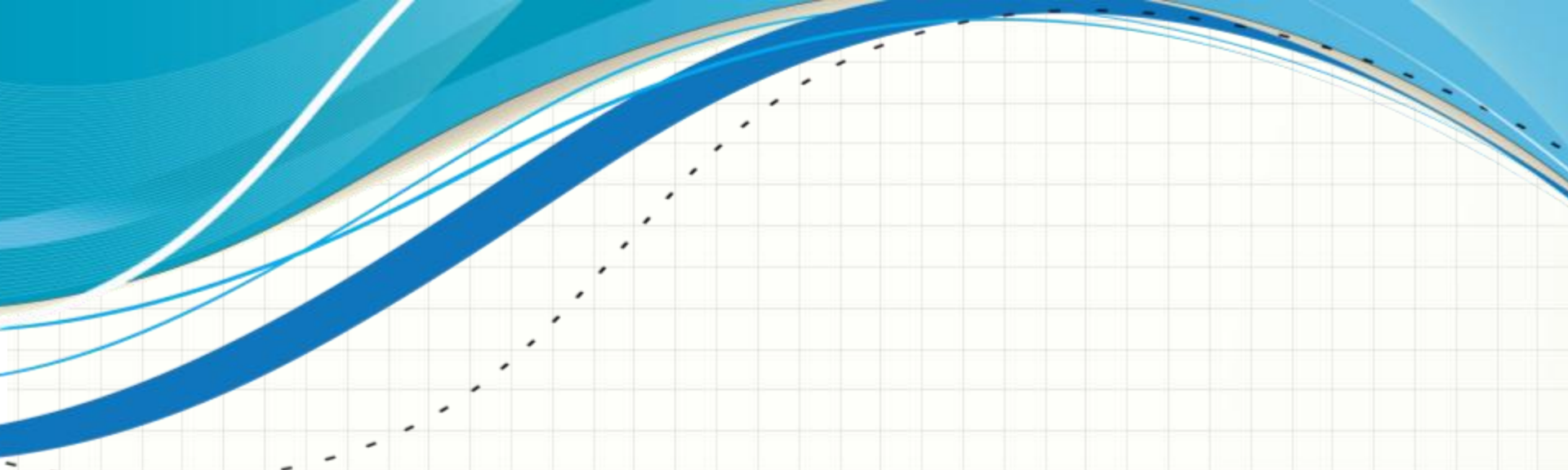
Other Two Level Implementation

$$F = XZ + Y'Z + X'YZ$$



$$F = (X+Z) (Y'+Z) (X'+Y+Z)$$





THANKS